

Turkish Legal RAG Final Technical Report

Students:

- Kutalmis Alp Fidan, 202211401
- Hasan Durmusoglu, 202111052

1. Project Scope

This project implements Turkish legal Question Answering with Retrieval-Augmented Generation. It compares Base RAG and Fine-tuned RAG under the same generator LLM family and supports evaluator-provided custom documents and custom benchmarks.

Dataset used in this project: **Datasets_Ceng493_legal_rag**.

2. Requirement Mapping

Requirement	Status	Evidence
Turkish legal RAG architecture	Done	src/legal_rag/pipeline.py
Base RAG vs Fine-tuned RAG with same LLM	Done	scripts/evaluate.py, scripts/gpu_rag_evaluate.py
Gold QA benchmark with 150-300 questions	Done	240 rows in gold_benchmark.json
Retrieval, QA, citation and faithfulness metrics	Done	Recall@k, MRR, nDCG, F1, ROUGE-L, citation hit, token support
Ablation study	Done	base, retriever, reranker, LLM style, full pipeline
GPU fine-tuning	Done	artifacts/gpu*/training_summary.json
Custom document support	Done	--custom-docs
Custom benchmark support	Done	--benchmark-file

3. Dataset Preparation

The dataset directory contains 7,579 corpus rows, 2,059 embedding training rows, 6,752 reranker training rows, 13,758 LLM instruction rows, a 240-question gold benchmark, and a 1,000-row RAG evaluation set. Corpus rows are loaded from **corpus.jsonl**. The benchmark stores **question**, **verified_answer**, and **gold_sources**; **source_id** and **corpus_row_id** are used as relevant document identifiers.

Preparation steps:

- Corpus rows are indexed with **id**, **title**, **text**, category, citation label, and source metadata.
- Embedding rows are converted into query, positive passage, and hard negative triples.
- Reranker rows are converted into query, candidate passage, and binary label pairs.
- LLM rows are converted into source-grounded instruction examples.
- Benchmark rows are kept separate from training and are used only for evaluation.

4. Selected Models and Finetuning Strategy

Component	Model	Finetuning strategy
Generator LLM	Qwen/Qwen2.5-3B-Instruct	LoRA/QLoRA SFT on grounded legal answer examples
Embedding	intfloat/multilingual-e5-base	Contrastive triplet training with hard negatives
Reranker	BAAI/bge-reranker-v2-m3	Binary classification over query-document relevance pairs

Base RAG and Fine-tuned RAG use the same generator model family. Fine-tuned RAG attaches a LoRA adapter to the same Qwen base model and uses the adapted retriever and reranker checkpoints.

GPU training was executed on NVIDIA GeForce RTX 4060 8GB.

Component	Steps	Average loss	Output
Embedding	300	0.0561	artifacts/gpu/embedding_model
Reranker	30	0.3735	artifacts/gpu/reranker_model
LLM LoRA	60	0.9744	artifacts/gpu/llm_lora

5. System Architecture and Metrics

The pipeline is: Question -> Retriever -> Candidate Documents -> Optional Reranker -> Context Builder -> Same LLM -> Answer with Citations.

Two implementations are provided. The lightweight pipeline uses pure Python BM25-style retrieval and local ablation weights for full-benchmark reproducibility. The GPU neural pipeline uses dense E5 retrieval, BGE reranking, and Qwen generation with optional LoRA adapter.

Metrics: Recall@1/3/5/10, MRR@10, nDCG@5/10, Exact Match, token F1, ROUGE-L, citation hit rate, top-1 citation accuracy, grounded answer rate, faithfulness token support, and unsupported sentence rate.

6. Full Gold Benchmark Results

The full 240-question gold benchmark was run with `scripts/evaluate.py`.

System	R@1	R@5	R@10	MRR@10	nDCG@10	F1	ROUGE-L	Citation hit	Faithfulness
Base	0.875	0.971	0.979	0.913	0.929	0.395	0.370	0.954	0.730
Fine-tuned	0.879	0.971	0.979	0.914	0.930	0.398	0.373	0.946	0.728

The fine-tuned pipeline gives a small gain in Recall@1, MRR@10, nDCG@10, F1, and ROUGE-L. Citation hit rate is slightly lower but remains high, so the improvement is reported as modest.

7. Ablation Study

All ablation rows were evaluated on the same 240-question gold benchmark.

System	R@1	R@5	MRR@10	F1	Citation hit	Faithfulness
Base	0.875	0.971	0.913	0.395	0.954	0.730
Adapted retriever only	0.879	0.975	0.912	0.392	0.942	0.727
Adapted reranker only	0.879	0.971	0.916	0.395	0.958	0.732
Adapted LLM style only	0.875	0.971	0.913	0.400	0.954	0.730
Full Fine-tuned RAG	0.879	0.971	0.914	0.398	0.946	0.728

The reranker-only row gives the best citation hit and faithfulness values, while the full fine-tuned row gives the best combined answer quality among adapted full-system settings.

8. GPU Neural Validation

GPU retrieval-only validation on the first 20 gold benchmark questions:

System	R@1	R@5	R@10	MRR@10	nDCG@10	Citation hit
Base neural RAG	0.60	0.85	0.85	0.67	0.713	0.90
Fine-tuned neural RAG	0.85	0.85	0.85	0.85	0.850	0.90

GPU generation smoke on one gold question:

System	R@1	F1	ROUGE-L	Citation hit	Faithfulness
Base neural RAG	0.00	0.250	0.219	0.00	0.375
Fine-tuned neural RAG	1.00	0.870	0.870	1.00	0.690

Embedding-only checks show mixed behavior: on the first 100 gold questions, Base E5 scored R@1 0.620 and Fine-tuned E5 scored R@1 0.520; on the first 200 RAG-eval questions, Base E5 scored R@1 0.865 and Fine-tuned E5 scored R@1 0.855. Therefore the report does not claim that the embedding checkpoint alone improves every metric.

9. Custom Data and Benchmark Support

The evaluator can provide a custom document collection and benchmark without changing code:

```
.\conda-envs\legal-rag-gpu\python.exe scripts\evaluate.py --custom-docs path\to\docs.jsonl --benchmark-file path\to\benchmark.json --artifacts artifacts --out reports\teache
```

Custom documents may be a directory of `.txt` files, a single `.txt` file, a `.jsonl` file with `id` and `text`, or a `.json` list of the same objects. Custom benchmark rows use `question`, `verified_answer`, and `gold_sources`.

The project includes `examples/custom_docs.jsonl` and `examples/custom_benchmark.json`. The custom input smoke test reached Recall@1 = 1.0, Citation hit = 1.0, and Top-1 citation accuracy = 1.0.

10. Error Analysis and Reproducibility

Observed failure modes include retrieval of nearby legal articles, strict Exact Match being too harsh for abstractive Turkish legal answers, overly long lightweight answers when neighboring articles overlap, and mixed embedding-only gains after short GPU training.

Main commands:

```
.\conda-envs\legal-rag-gpu\python.exe scripts\train_lightweight.py --dataset Datasets_Ceng493_legal_rag --out artifacts
.\conda-envs\legal-rag-gpu\python.exe scripts\evaluate.py --dataset Datasets_Ceng493_legal_rag --artifacts artifacts --benchmark gold --out reports\evaluation_results_gold
.\conda-envs\legal-rag-gpu\python.exe scripts\gpu_rag_evaluate.py --dataset Datasets_Ceng493_legal_rag --benchmark gold --limit 20 --no-generation --out reports\gpu_rag
.\conda-envs\legal-rag-gpu\python.exe scripts\gpu_rag_evaluate.py --dataset Datasets_Ceng493_legal_rag --benchmark gold --limit 1 --out reports\gpu_rag_generation_gold
.\conda-envs\legal-rag-gpu\python.exe -m unittest discover -s tests -p "test_*.py"
```

The latest unit test run passed 3 tests. The project satisfies the core assignment requirements: Turkish legal RAG, same-LLM Base vs Fine-tuned comparison, gold benchmark evaluation, ablation study, GPU fine-tuning artifacts, custom document support, custom benchmark support, and source-grounding analysis.